

Tel Aviv University
The Iby and Aladar Fleischman Faculty of Engineering

Electronics - Laboratory B
Experiment FPGA – Stopwatch

Designed By:

Mohamad Jundia - 214234478

Abed Elfatah Mansour - 213144484

2025-2026

1. Full Adder:

a)

Done.

b)

The truth table of the FA:

a	b	Ci	Co	sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

We can use SOP to come out with the logic functions for both outputs. But we chose to use the reduced (known) function from the digital logic systems course.

The rows that produce Co are marked in yellow.

$$\text{sum} = a \oplus b \oplus Ci, \quad Co = (a \cdot b) + (Ci \cdot (a \oplus b))$$

c)

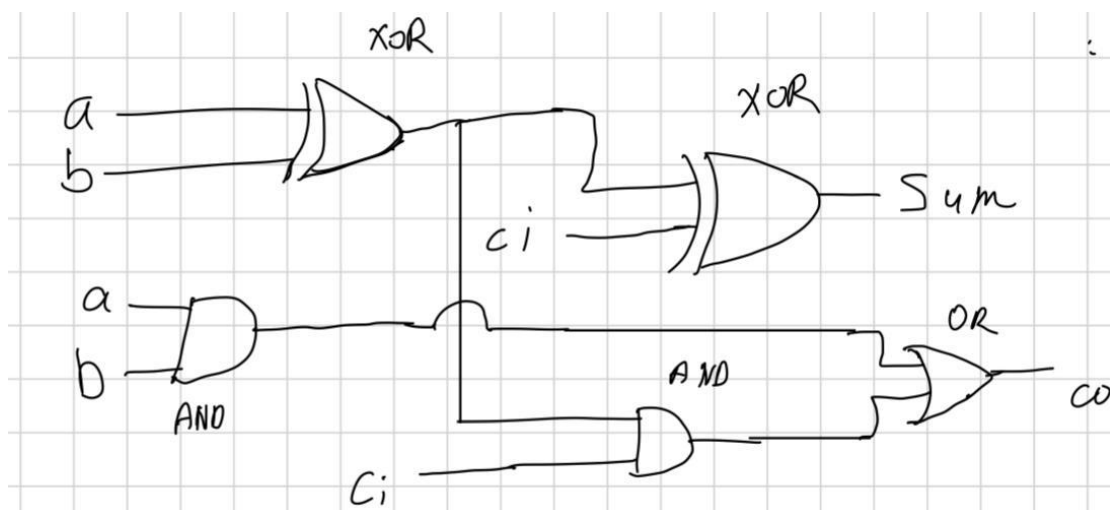
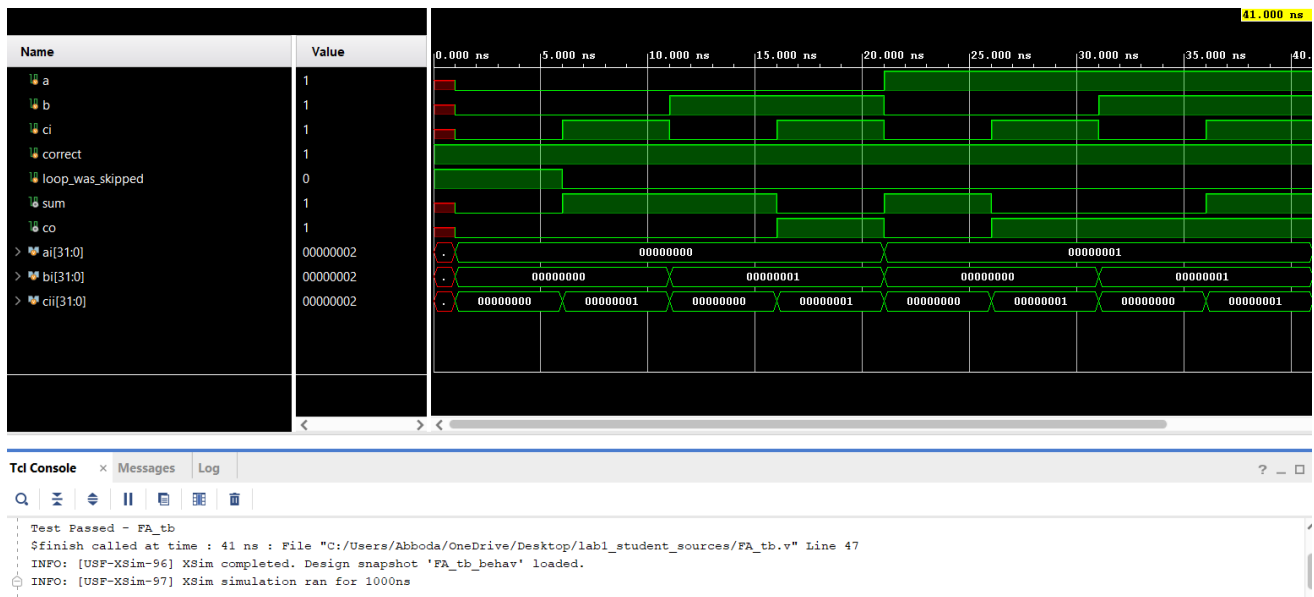


Figure 1 – FA Block Design

e+f)



We used the test bench to check every possible input for the Full Adder. The simulation showed the correct outputs for every case, and the console confirmed the design works by displaying “Test Passed”.

2. CSA:

A)

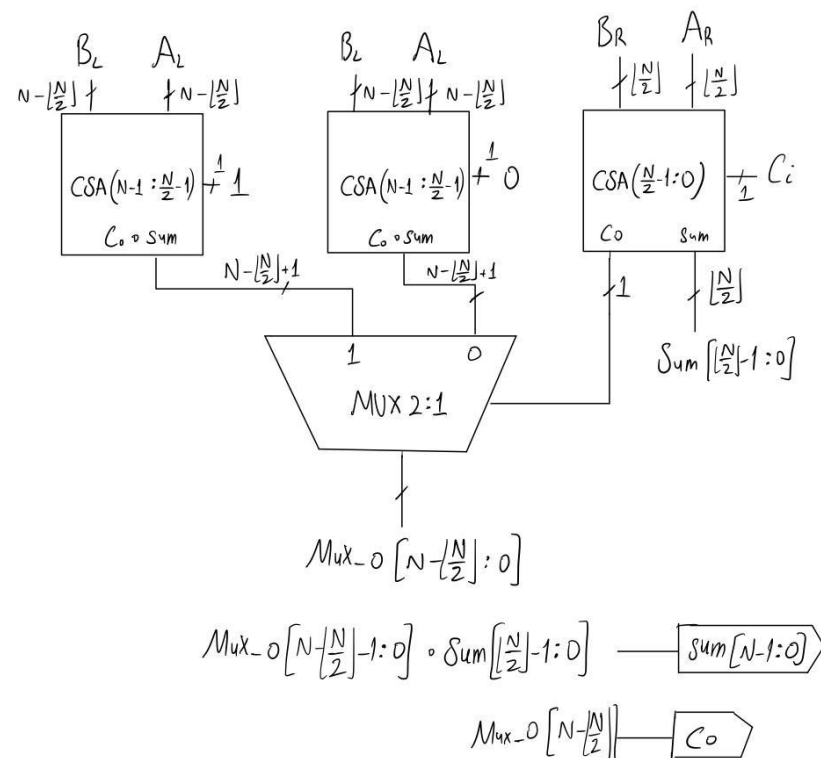


Figure 2 – CSA Block Design

We have designed CSA(N) recursively using divide-and-conquer implementation. We split the N-bit adder into 3 blocks, one block that compute the lower half, the remaining blocks (2 blocks) compute the upper half in parallel, one of them assuming that carry_in is 1 and the other block assuming that carry_in is 0. Finally, a mux selects the correct upper result based on the lower carry.

B) Done.

C) A Conditional Sum Adder is faster than a ripple-carry adder because it doesn't wait for the carry to move through every bit. Instead, it calculates the possible sums in parallel for both possible carry_in values (0 and 1) and then simply selects the correct result once the actual carry is known. This greatly reduces the overall delay, especially for large adders.

A ripple carry adder can be a better choice when the number of bits is small, or when saving area and power is more important than speed, since it is easy to design.

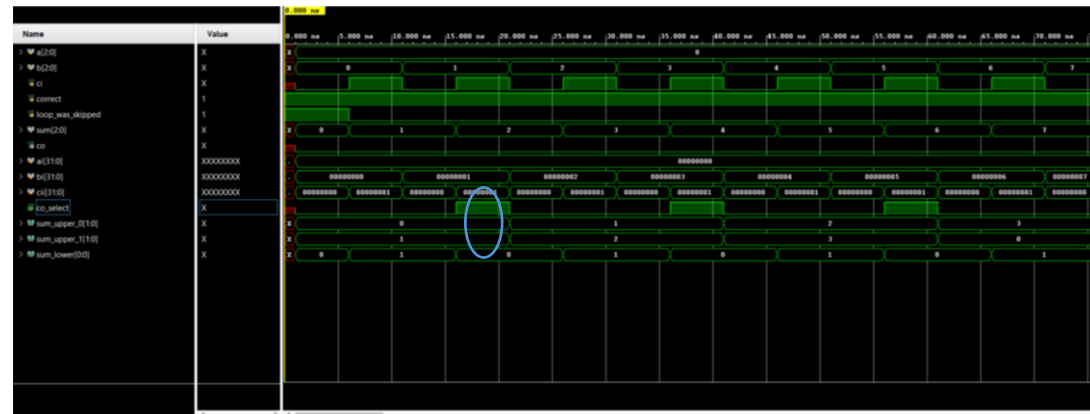
D) a generate block is a Verilog construct that lets us Instantiate the module design for any parameter N. without the "generate block", we would write separate modules for each bit width, which is impractical and unnecessary.

We need a generate block in a Conditional Sum Adder because the CSA is built recursively, the adder is repeatedly split into smaller adders whose sizes depend on the parameter N so generate block allows us to create CSA instances for any bit width.

E) Done.

F+G)

The testbench for CSA(3): **N=3**

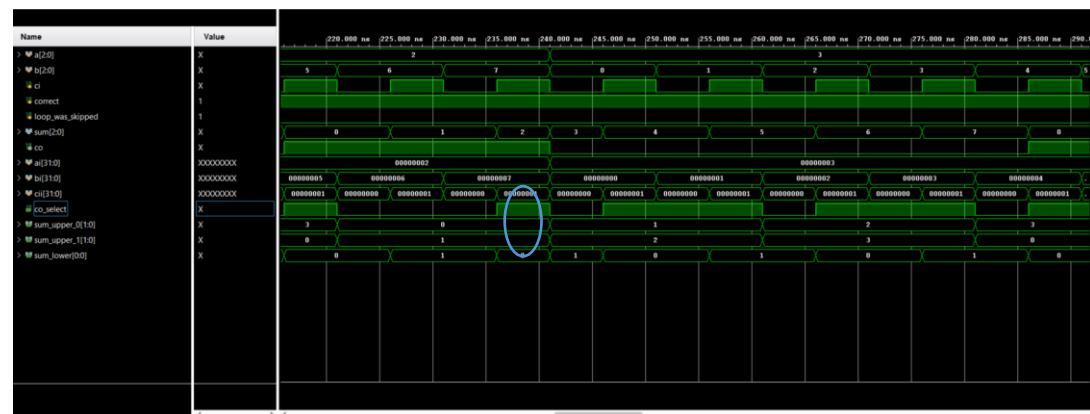


Td Console: Messages Log

```

INFO: [WaveData 42-604] Simulation restarted
run 10 ns
Test Passed - CSA_tb
$finish called at time : 641 ns : File "C:/Users/Abhoda/OneDrive/Desktop/lab1_student_sources/CSA_tb.v" line 50

```

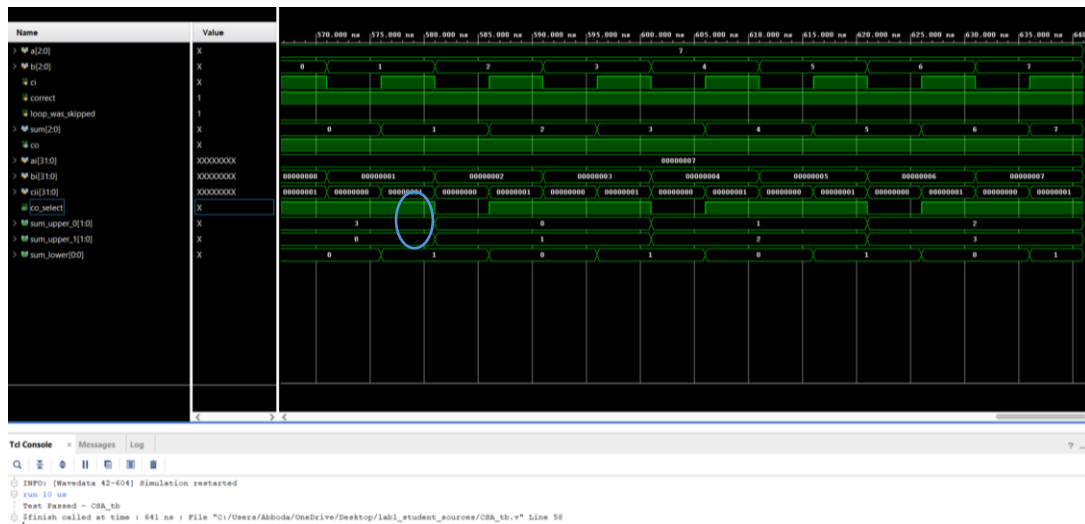


Td Console: Messages Log

```

INFO: [WaveData 42-604] Simulation restarted
run 10 ns
Test Passed - CSA_tb
$finish called at time : 641 ns : File "C:/Users/Abhoda/OneDrive/Desktop/lab1_student_sources/CSA_tb.v" line 50

```

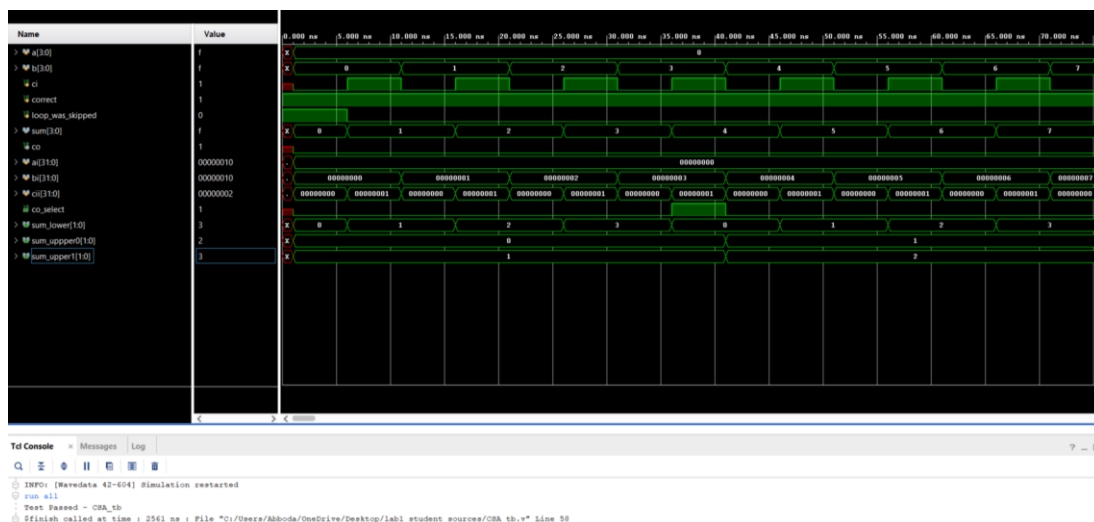


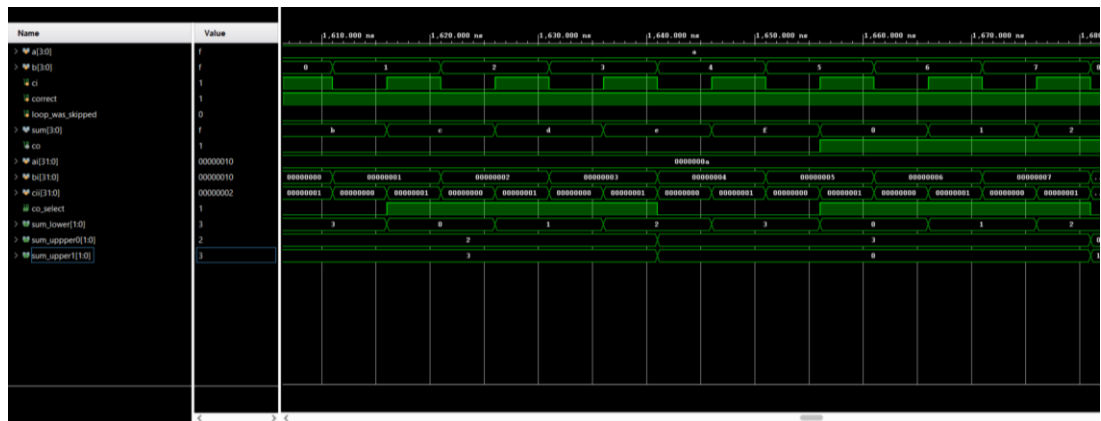
We used a testbench to verify the CSA module by checking all possible input combinations. The simulation showed the correct outputs for every case, and the console confirmed that the design works correctly by displaying “Test Passed”. We can see that the signal named `co_select` serves as the select bit of the MUX that chooses the upper-half sum. The simulation confirms that this signal correctly implements the conditional behavior of the design. (we marked it in on the waveform)

If we considered the last waveform image:

$a[2:0] = 3'd7$, $b[2:0] = 3'd1$, $ci = 1'b1$. Applying the divide and conquer implementation leads to $sum_lower = 1'b1$, $sum_upper_0[1:0] = 2'd3$, $sum_upper_1[1:0] = 2'd0$. According to the waveform, the carry-out from the lower half is 1, which selects the $sum_upper_1[1:0]$ result computed with $carry_in = 1$. Therefore, the final sum $sum[3:0]$ is formed by concatenating $sum_upper_1[1:0]$ with sum_lower , resulting in the binary value 001, and the final carry-out is 1, as expected.

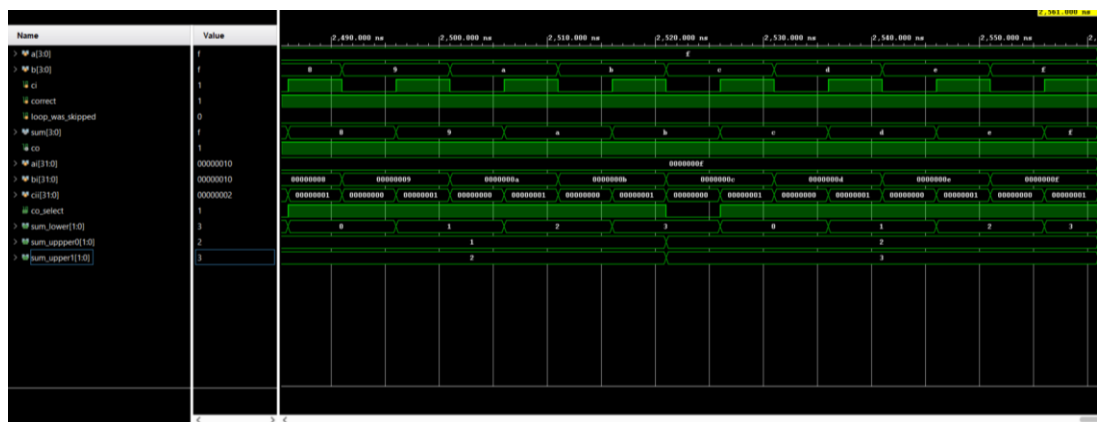
The testbench for CSA(4): **N=4**





Td Console Messages Log

INFO: [WaveData 42-604] Simulation restarted
run all
Test Passed - CBA_tb
Finish called at time : 2561 ns : File "C:/Users/Abhoda/OneDrive/Desktop/lab1_student_source/CBA_tb.v" line 58



Td Console Messages Log

INFO: [WaveData 42-604] Simulation restarted
run all
Test Passed - CBA_tb
Finish called at time : 2561 ns : File "C:/Users/Abhoda/OneDrive/Desktop/lab1_student_source/CBA_tb.v" line 58

3. Limited Incrementor

A)

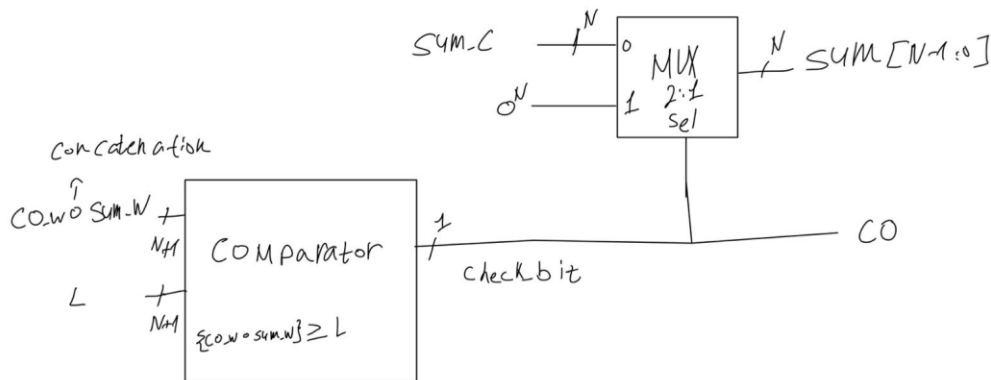
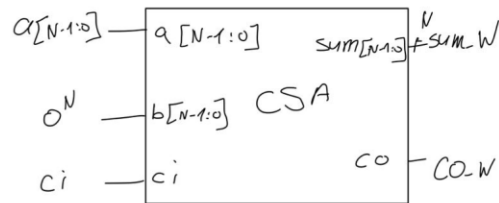
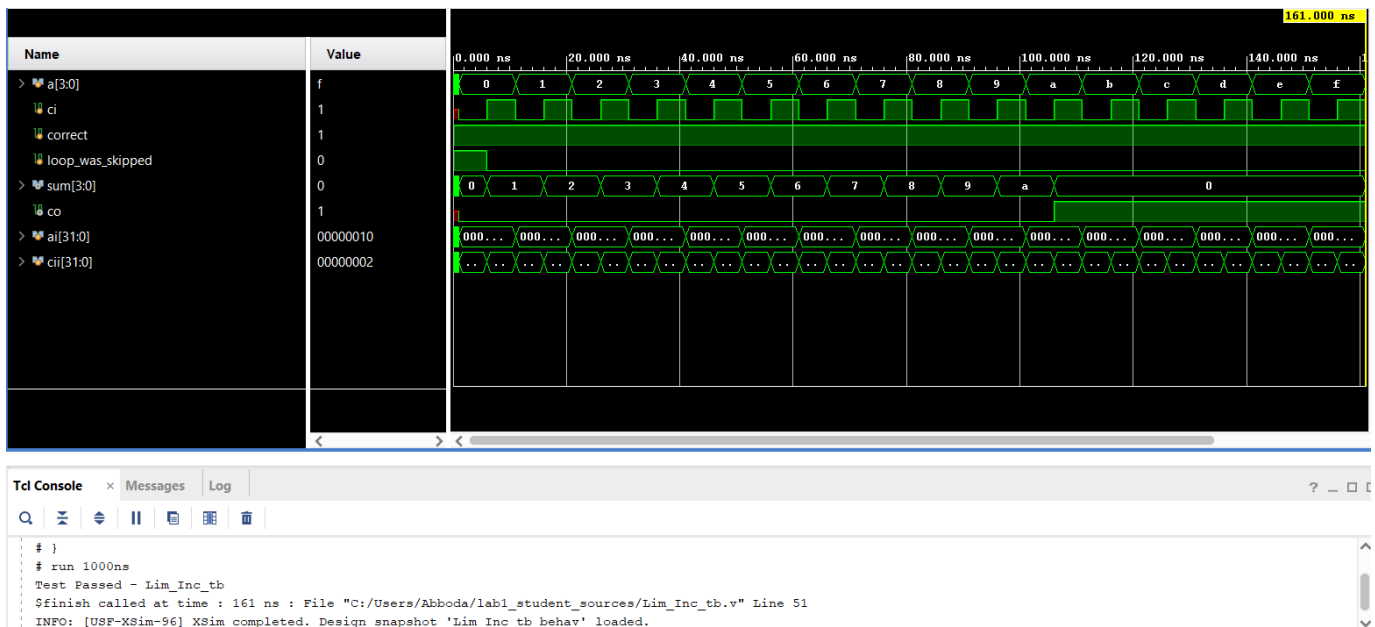
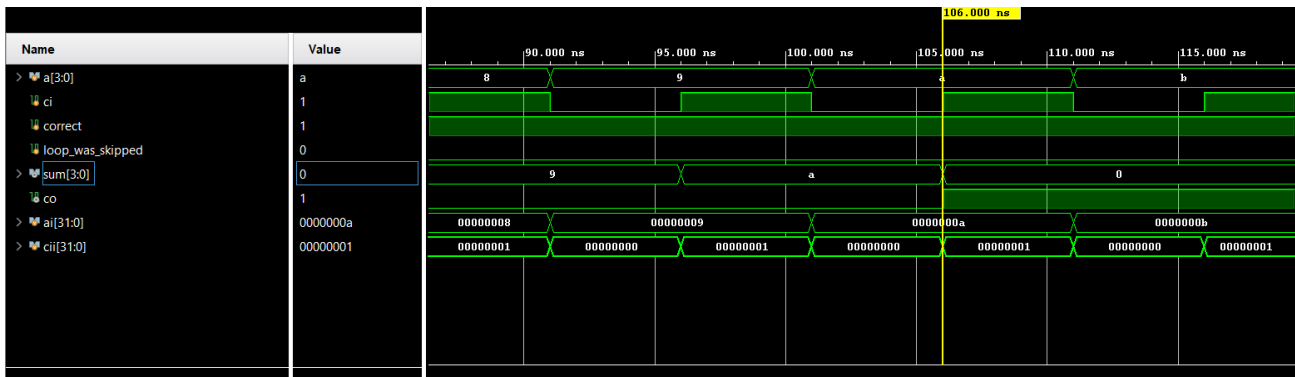


Figure 3 – Lim_enc Block Design

D)

as provided in the updated mail, we design the testbench with $L=11$ so the expected input "a" should be 4 bits long. The test passed as we can see:





As shown by the yellow cursor in the waveform, the Co signal is asserted for the first time at this moment. This behavior is expected and verifies the design specification where the limit parameter L is set to 11 (which is b in hexadecimal).

At this instance, the input a is 10 and the Ci is 1, resulting in a raw sum of 11. Since the calculated sum is greater than or equal to the limit, the module correctly outputs a Co of 1 and resets the sum output to 0.

The raw sum differs from the final sum whenever the limit is reached or exceeded. In the specific case marked by the cursor, the raw sum calculated by the adder is 11. However, because this value meets the limit, the logic forces the final sum output to 0. Therefore, the raw sum is 11 while the final sum is 0.

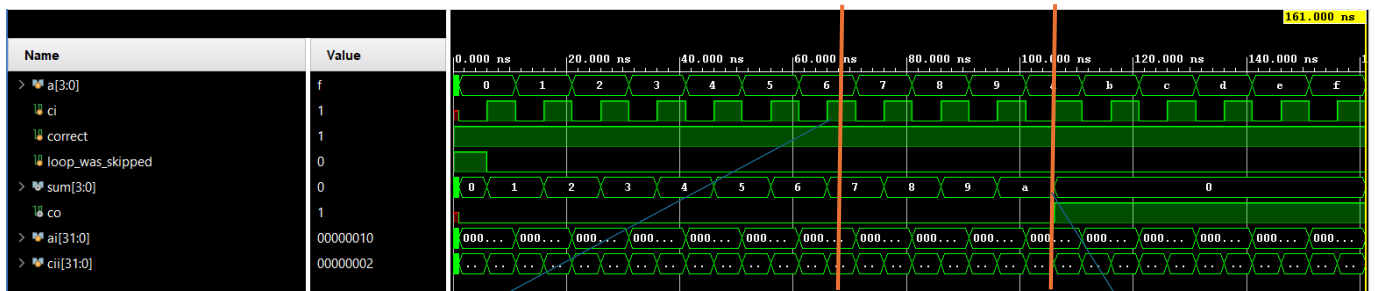
E)

The raw sum and the final output differ because the raw sum is a pure addition result, while the final output is restricted by a specific limit. When the total hits this limit, the logic forces the final output to zero. Therefore, whenever the limit is reached, the raw sum remains high but the final output resets, causing the two values to differ.

The only time they never differ is when the limit matches the full capacity of the hardware bits ($L = 2^N$) where N is the number of bits. In this case, when the count gets full and increments, the hardware naturally runs out of room and wraps around to zero. Since the logic would also force a zero at this exact moment, the natural result and the forced result are identical.

For this specific situation, the complex comparison logic is unnecessary. A simpler design is a standard binary adder. We can rely on the adder's natural behavior to wrap around to zero automatically. The standard carry-out signal can then be used to indicate that the cycle is complete, achieving the same goal with less hardware.

F)



For example, here we can see that $a=6$ and $ci=1$. The sum is $6+1=7$. Since $7 < 11$, we can see that the output did update to 7 as expected with $co=0$. This confirms the normal operation when the limit is not reached.

In this simulation, the input a is 10. Adding 1 results in 11, which matches the limit. Consequently, the waveform shows that the output stays at $sum=0$ and $co=1$ since we passed the limit, verifying that the overflow logic works correctly.

4. Counter:

A)

The output `time_reading[7:0]` represents a time range of 0 to 99 seconds. This is because the 8-bit signal is essentially split into two 4-bit BCD counters: the lower bits (0-3) counts the unit seconds from 0 to 9, and the upper bits (4-7) counts the tens of seconds from 0 to 9. Therefore, the total value ranges from 00 up to a maximum of 99 before wrapping back to zero.

B)

To measure exactly one second, we utilize a `Lim_Inc` instance with the limit set to $L=100M$. Given the system clock period of 10ns, 100 million clock cycles correspond to exactly one second ($100M \cdot 10n = 1s$). When this count is reached, the module asserts the `co` signal. This signal serves as the enable trigger, indicating to the main counter that it is time to update the seconds digit.

C)

The number of `Lim_Inc` modules is determined by the number of display digits plus one. We need one module for each digit (one for the units and one for the tens) to handle the actual time counting. Additionally, we require one extra module to act as the clock divider, this instance counts the system clock cycles to generate the exact 1-second trigger. Therefore, the total number of instances needed is the number of digits + 1.

D)

To generate a 0.1 Hz tick, we first determine the period of the signal, which is $1 / 0.1\text{Hz} = 10$ seconds. We then divide this target duration by the system clock period to find the required count. With a target of 10s and a clock of 10ns, the calculation is $10s / 10ns = 1,000,000,000$. Therefore, we set $L = 1,000,000,000$ to ensure the `co` signal asserts exactly once every 10 seconds.

E)

If the input clock changes to 50MHz which has a period of 20ns, our design connections remain exactly the same. Our design consists of three incrementers, and the parameters for the second and third instances (the seconds and tens digits) do not change because they count logical digits (0-9), not time. The only thing we must change is the first incrementer, which is responsible for generating the 1-second tick. We need to update its limit parameter L to match the new frequency: $L = 50 \text{ MHz} \times 1\text{sec} = 50,000,000$. We lower the limit from 100M to 50M because the clock is slower, so it takes fewer cycles to reach one second.

F)

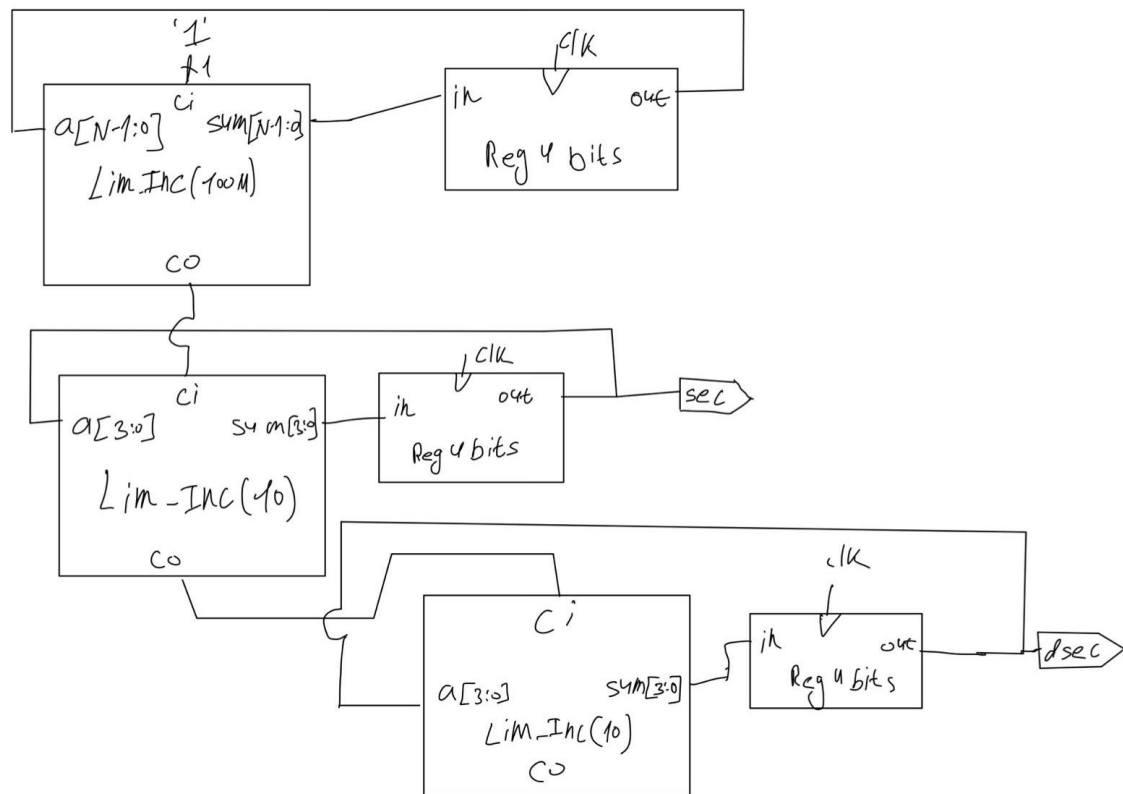


Figure 4 – Counter Block Design

The counter functions by pairing a register with combinational logic in a synchronous feedback loop. The register holds the current value, which continuously feeds into the Lim_Inc modules to calculate the next state. When the enable signal is active, the logic increments the count or resets it to zero if the limit is reached. The register updates to this new value only on the rising edge of the clock. To handle multi-digit numbers, the overflow signal from one stage is used to enable the next stage, creating a cascade effect.

H)

Done.

I)

This testbench generates the clock and control signals to verify the counter's operation in real seconds. It confirms that the lower nibble counts 0-9 and the upper nibble increments exactly when the lower wraps, validating that the cascading carry logic works correctly.

Additionally, the testbench uses synchronization(sync) to ensure inputs are driven relative to the active clock edge, this prevents race conditions and ensures that the signals are stable before the counter samples them.

Here we can see that the module passed the test.

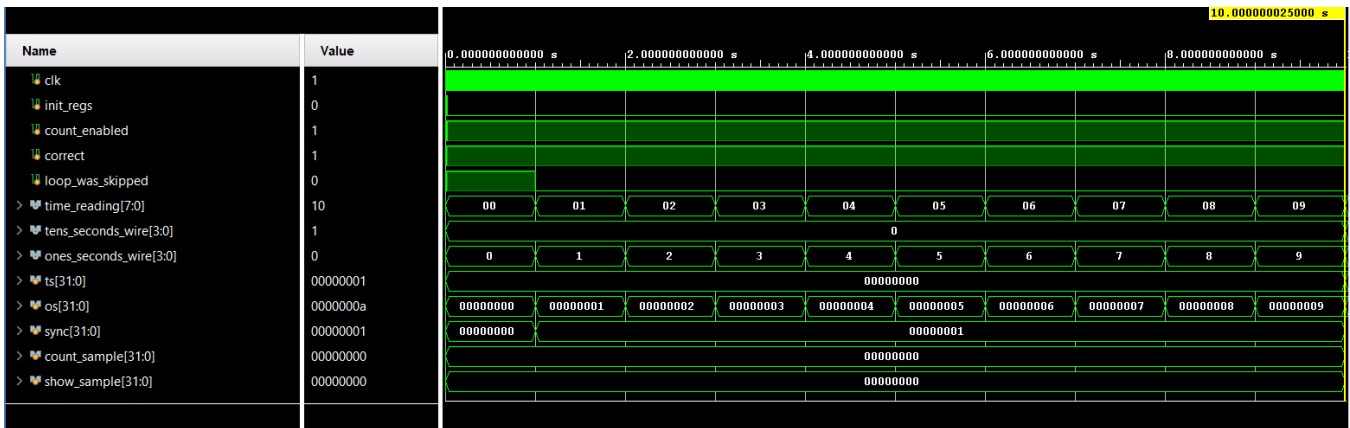
```

Time 0 9 Verified.
Test Passed - Counter_tb
$finish called at time : 10000000025 ns : File "C:/Users/Abboda/lab1_student_sources/Counter_tb.v" Line 72
run: Time (s): cpu = 00:18:50 ; elapsed = 00:30:23 . Memory (MB): peak = 2478.070 ; gain = 0.000

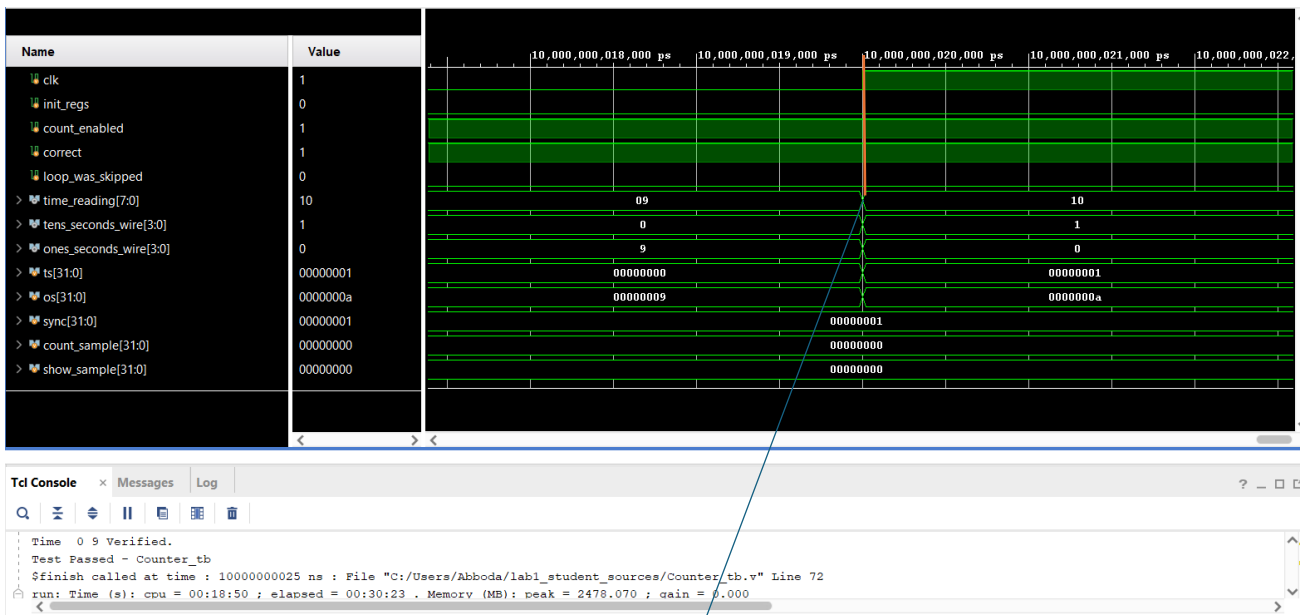
```

))

The full simulation:



The simulation ran for exactly 10 seconds, confirming the correct real-time duration. The waveform shows time_reading matching the internal tens and ones wires, validating the digit transitions. Additionally, the sync variable confirms that the testbench checks remained properly aligned with the clock edges throughout the test.



The zoomed-in waveform captures the critical transition at the 10-second mark, where time_reading updates from 09 to 10. This demonstrates the cascading logic: as the ones_seconds_wire wraps from 9 to 0, it successfully triggers the tens_seconds_wire to increment from 0 to 1.

K)

The Lim_Inc module calculates the next number but has no memory to store it. We use an external register to hold the current count so that updates happen only when the clock ticks. This keeps the math separate from the memory, ensuring the timing is perfectly controlled.

L)

in our design the maximum time is 99 as explained before, then it rolls back to zero and starts counting again.

M)

To display MM:SS, the design requires five modules: the clock divider (generating the 1Hz tick) driving four cascading Lim_Inc instances. The output bus must be increased to 14 bits (4 bits per ones digits and 3 for tens digits). We would cascade these modules similarly to the original design, where the overflow of a lower digit acts as the enable signal for the next. In addition, the Seconds_Tens and Minutes_Tens modules require a limit of L=6 to enforce the correct modulo-60 timing.

5.Stash:

A)

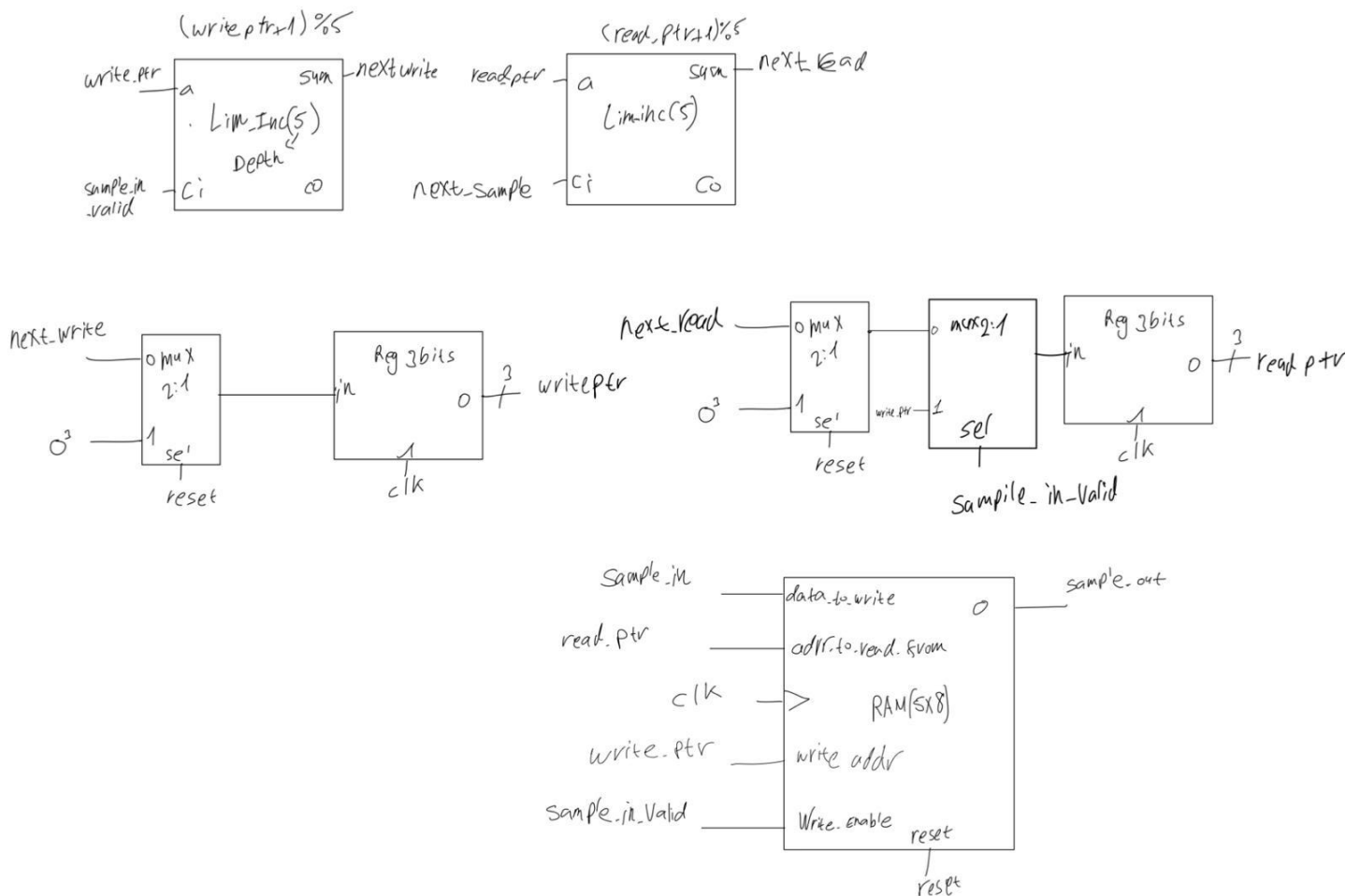


Figure 5 – Stash Block Design

Everything excluding the RAM, the design focuses on the implementation of the write and read address pointers. The RAM itself is implemented using five 8-bit registers for data storage. This memory block is designed to write new data only when the write-enable signal is asserted, while continuously outputting the data stored at the current read address.

B+C) Done.

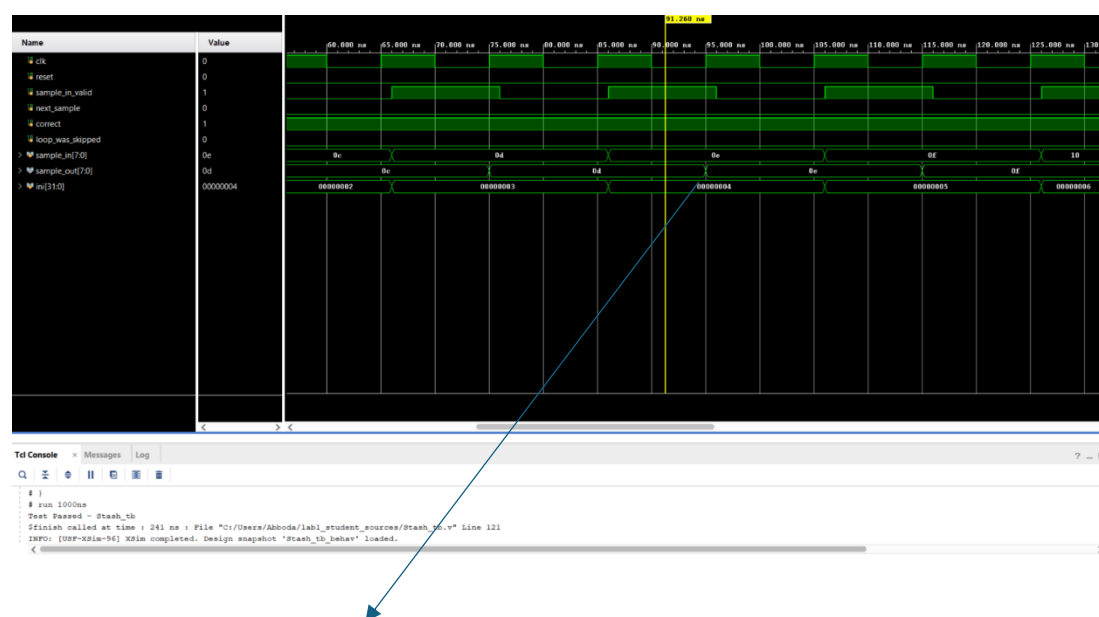
D)

The Stash_tb verifies the module using three specific tests. First, it writes more data than the memory can hold (7 values into 5 slots) to prove that the write pointer correctly wraps around to the beginning and that the output immediately jumps to show the new data. Second, it manually steps through the memory to ensure the read pointer correctly cycles from the last slot back to the first. Third, it triggers a new write and a next read command at the exact same time to confirm that the system correctly prioritizes showing the new data. If all checks pass without any errors, the simulation prints "Test Passed".

As we can see the design passed our TB:

```
# }
# run 1000ns
Test Passed - Stash_tb
$finish called at time : 241 ns : File "C:/Users/Abboda/lab1_student_sources/Stash_tb.v" Line 121
INFO: [USF-XSim-96] XSim completed. Design snapshot 'Stash_tb_behav' loaded.
```

E)



In this cycle where the yellow cursor is, sample_in_valid is high and next_sample is low, the waveform shows sample_out immediately updating to match the new input on the very next clock edge. This occurs because the Jump Logic detects the valid input and overrides the standard incrementing behavior, forcing the read pointer to align directly with the write pointer.

F)

The output selection logic functions like a MUX controlled by the `read_ptr`. Using a continuous assignment (`assign sample_out = stash_mem[read_ptr];`), it selects one specific register from the memory array and routes its data directly to the `sample_out` port. While the pointer itself changes only on the clock edge, the data connection is purely combinational and instant. This ensures that the moment the pointer updates to a new address, the correct value appears at the output without any additional delay.

G)

An alternative approach to the current "Pointer Jump" logic is to use a MUX. Instead of forcing the read pointer to move to a new address inside the memory, this method places a selection switch at the very end of the circuit. If new data arrives, this switch routes the live input directly to the output:

```
assign sample_out = (sample_in_valid) ? sample_in : stash_mem[read_ptr];
```

This logic effectively bypass the memory storage for that moment. The main advantage of this approach is that it offers zero latency, allowing the user to see new data instantly without waiting for a clock tick, while also preserving their current spot in the memory. However, the downside is that this direct connection is less stable; any electrical noise on the input wire will appear immediately on the output, whereas our current design filters out glitches by waiting for the steady clock signal to update the pointer.

6. Control:

A) The FSM shown in Figure 1 is presented as a Mealy FSM, where each output depends on both the inputs and the current state. However, it can also be represented as a Moore FSM, since the output `count_enabled` is 1 only in the counting state, and the output `init_regs` is 1 only in the idle state. Therefore, in this case, the output depends only on the current state. Furthermore, the outputs of the control module satisfy:

```
init_regs    = (state == IDLE);  
count_enabled = (state == COUNTING);
```

B)

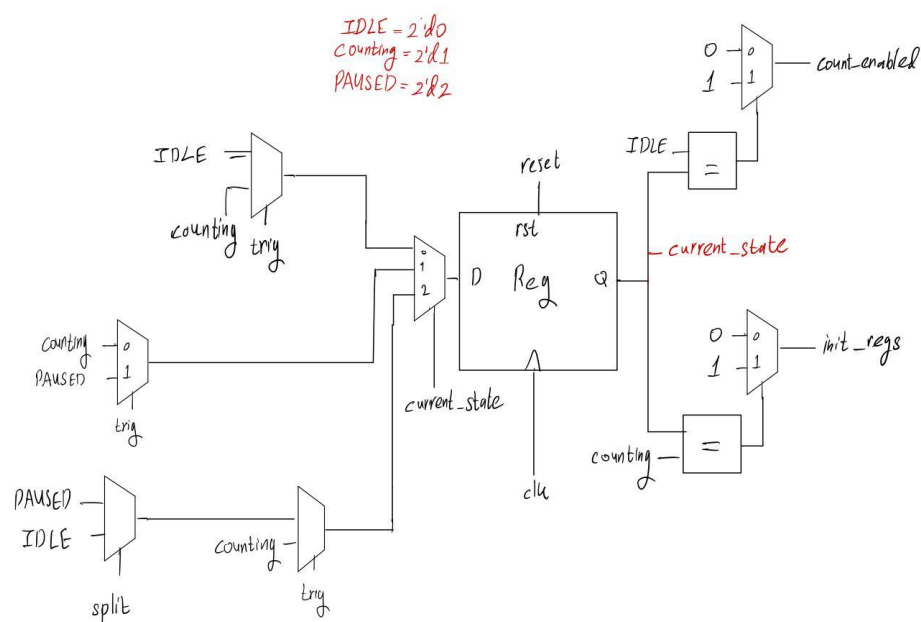
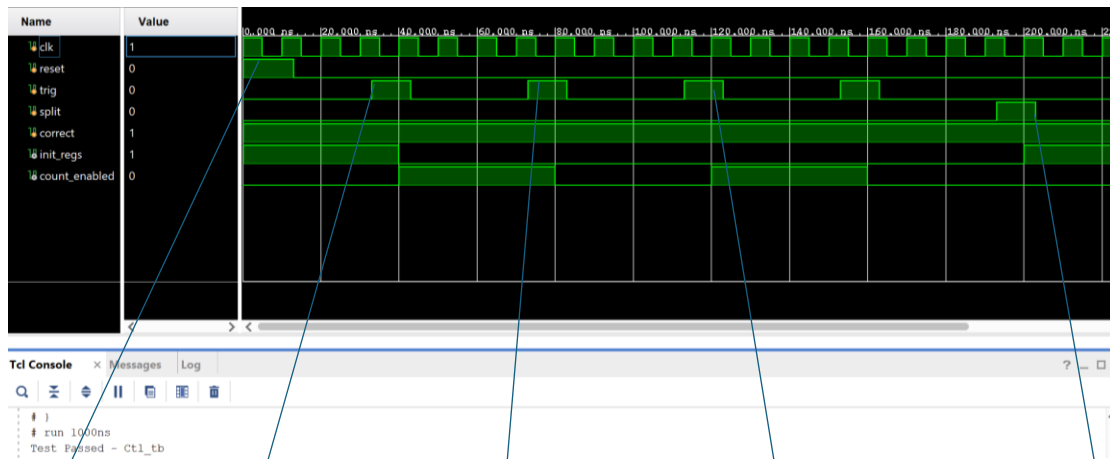


Figure 6 – Contol Block Design

In the block diagram, we used a comparator block to check for equality, one instance compares the current state to the COUNTING state, and another instance compares the current state to the IDLE state.

C+D) Done.

E+F) To verify the functionality of the Control FSM, we implemented a testbench that ensures that the FSM correctly transitions between all three possible states (IDLE, COUNTING, PAUSED) and asserts the correct output control signals. The waveform of the simulation:



Reset signal assert that init_regs = 1, count_enabled = 0

Trigger pulse, the FSM transitions to counting state, init_regs drops to 0 and count_enabled becomes 1.

Trigger pulse, the FSM transitions to pausing state, count_enabled drops to 0 and init_regs becomes 1.

Trigger pulse, the FSM transitions to counting state, resume the counting operation.

Split pulse that stops the counting operation and reset the system, which count_enabled equals to 0 and init_regs becomes 1.

7. Debouncer:

A)

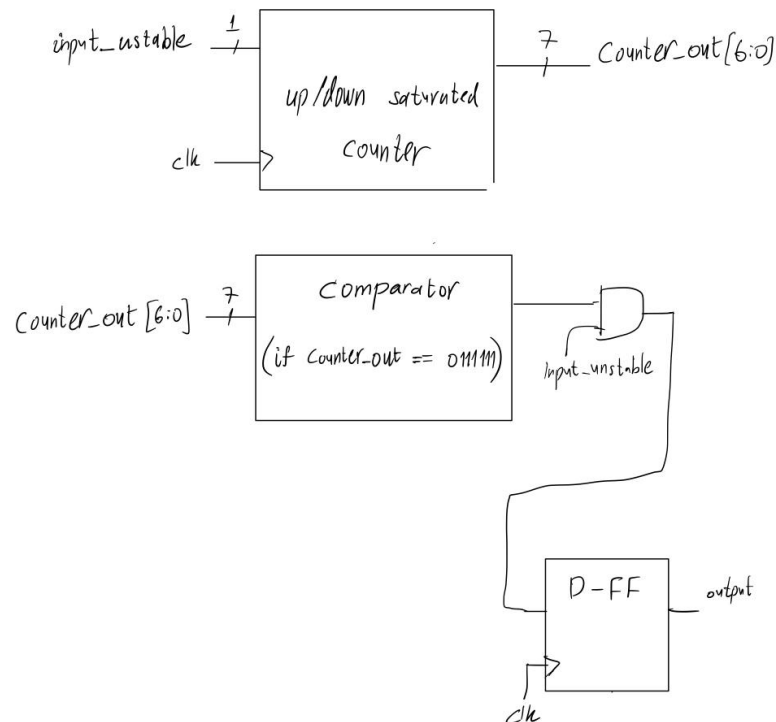


Figure 7 – Denouncer Block Design

B+C) Done.

D) Choosing a high counter value, which corresponds to a larger **COUNTER_BITS** parameter, means that the input signal must remain stable for a longer period of time before a button press is recognized. This provides strong immunity to noise and effectively suppresses mechanical contact bouncing. reducing the **COUNTER_BITS** parameter allows the debouncer to respond more quickly, decreasing the delay between a button press and the corresponding system action. While this improves responsiveness, it also makes the system more sensitive to noise and bouncing, increasing the risk that a single physical press may be detected as multiple presses.

This tradeoff is related to the low-pass filter debouncing approach, a larger counter value in the digital debouncer is analogous to a lower cutoff frequency in a low-pass filter more aggressive filtering, better noise suppression, but slower response. Conversely, a smaller counter value corresponds to a higher cutoff frequency, allowing faster changes to pass through but providing less filtering of bounce.

8. Seven Segment Display:

A) Done.

B) We have added the Verilog file of the 7 segment to the project, so we choose the following indexes of clkdiv[19:0]:

```
s = clkdiv[19:18];
```

C) We can see that clkdiv[19:0] is a counter of 20 bit that determines how long we stay on one digit. In order to meet the requirements, we are required to make a refresh time of between 16ms and 1ms for a human eye to see, so we choose the indexes 18 and 19.

The LSB of s (bit 18) toggles every 2^{18} clock cycles, given a $100[MHz]$ it can be said that:

$$T_{digit} = 2^{18} * T_{clk} = 2.62[ms]$$

Calculating the total refresh time, we have 4 digits so:

$$T_{total} = 4T_{digit} = 10.485[ms]$$

We can see that the total refresh time meets the requirements: $1[ms] < T_{total} < 16[ms]$

9. Stopwatch:

Block Diagram:

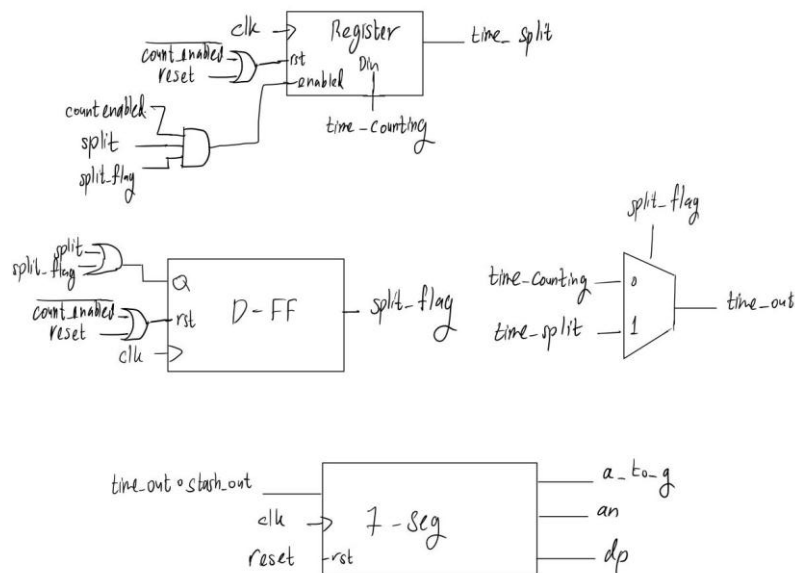
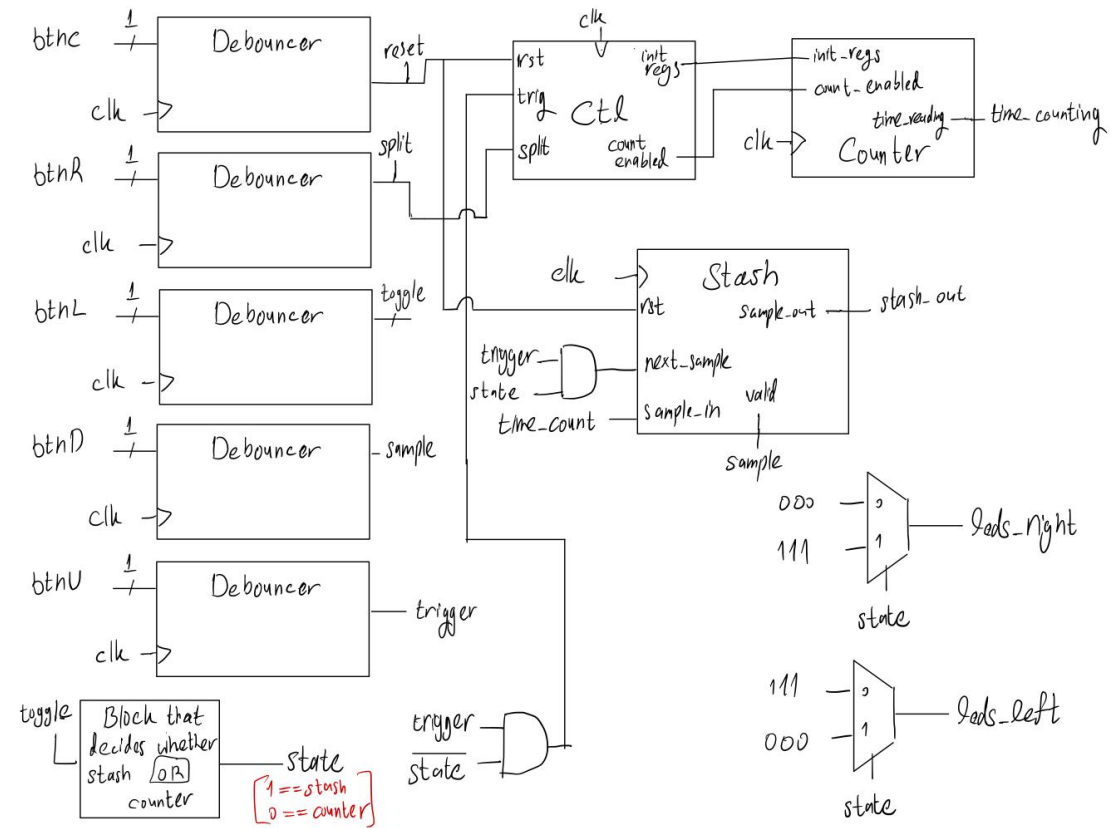
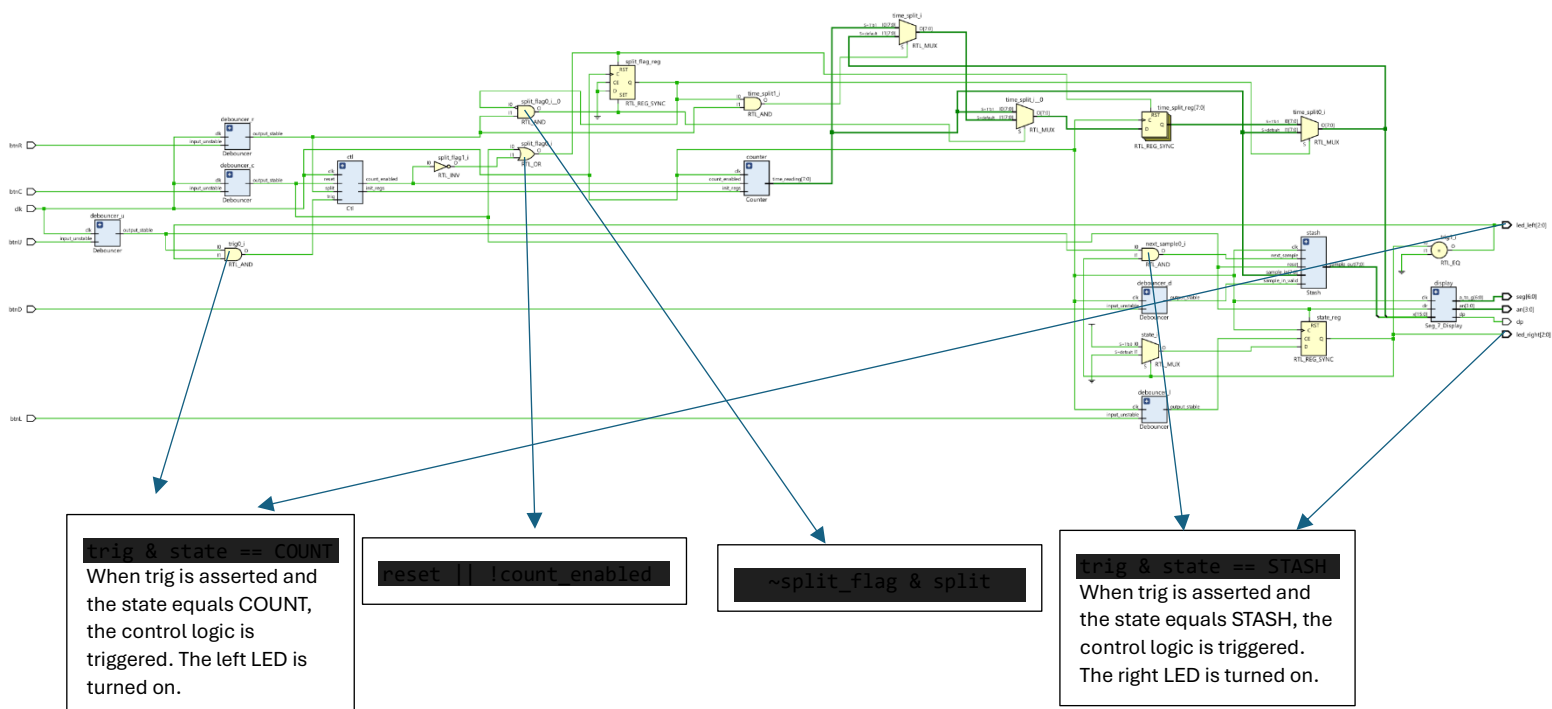


Figure 8 – Stopwatch Block Design

In order to support 5 buttons, we instantiated five debouncer modules one for each button. We used a block that determines the current system state (stash or counter). The overall design includes a single instance of the stash module, a single instance of

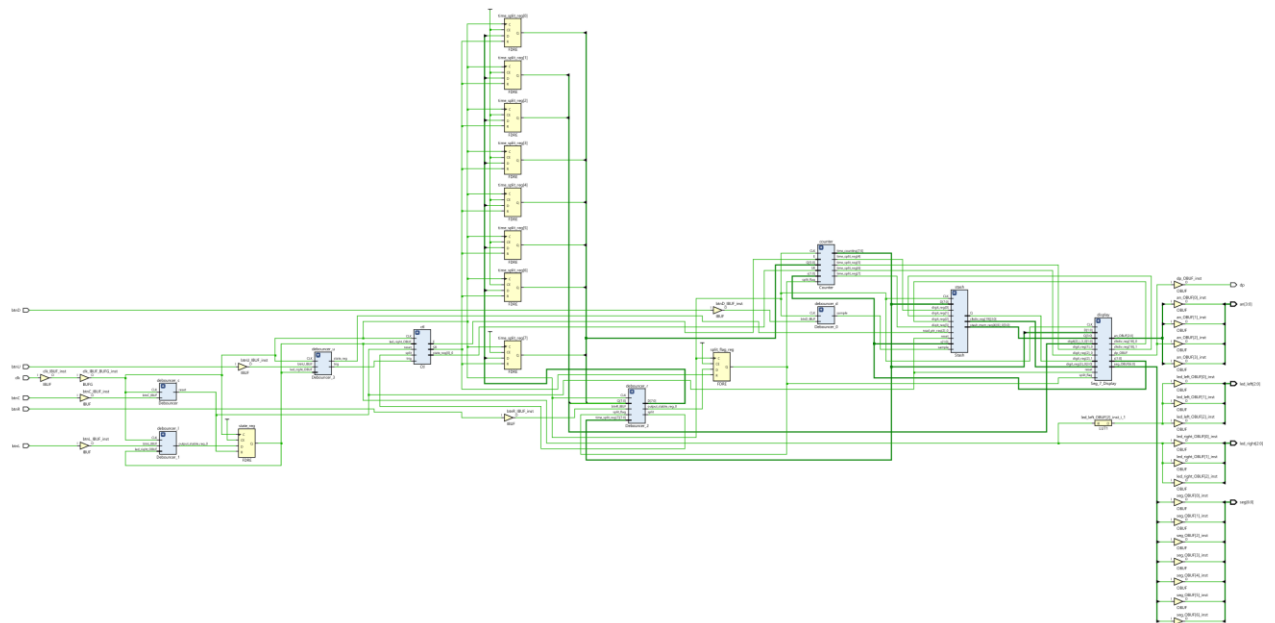
the control module, and one instance of the 7 segment display module, all working together to manage system behavior and output display.

The elaborated design schematic:



We can see that the elaborated design schematic accurately represents the Verilog implementation of the stopwatch module.

The synthesized design schematic:



The synthesized design is the result of translating out Verilog RTL code into the specific hardware resources on the FPGA. It shows the actual physical components that will be

used inside the chip. In the elaborated design, inputs and outputs were represented as lines. However, in the synthesized design, input and output buffers have been added to the ports to manage the electrical interface with the board. Additionally, while previous design has displayed standard logic (AND, NOT, OR), in the synthesized design all combinational logic mapped to LUTs (It stands for "look up table" that stores the truth table of a specific functions).

10. Setting up constraints:

We set up the design constraints in the `lab1_constraints.xdc` file as requested. This included uncommenting the lines for the system clock (pin W5) and establishing the required timing constraints. We also mapped the physical input/output interfaces, connecting the five buttons (btnC, btnU, btnR, btnL, btnD), the seven-segment display lines (seg, an, dp), and the status LEDs (led_left and led_right) to the appropriate pins on the FPGA.

11. Implementing the design:

A screenshot of the synthesized design schematic for the top-level module is provided in section 9 (Stopwatch).

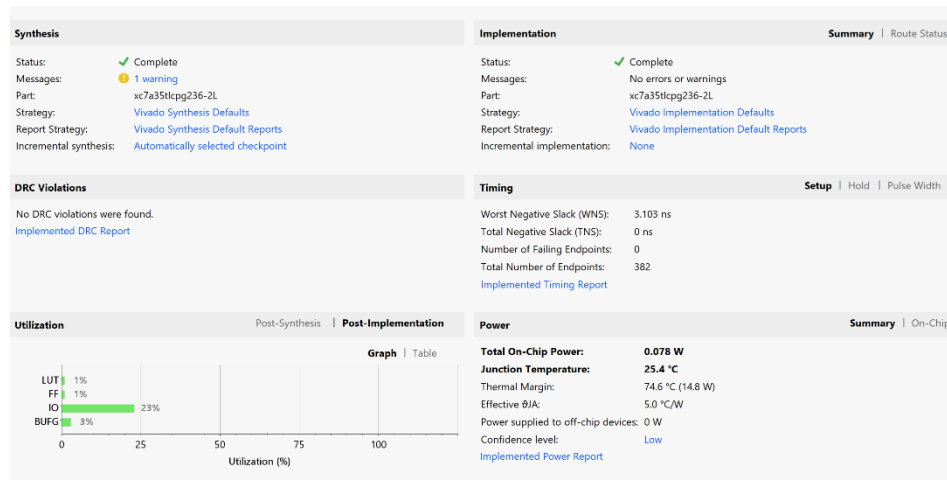
a screenshot of the design timing summary:

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 3.103 ns	Worst Hold Slack (WHS): 0.249 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 382	Total Number of Endpoints: 382	Total Number of Endpoints: 159
All user specified timing constraints are met.		

We can see that the worst negative slack (WNS) is 3.103[ns]. Since the clock frequency is 100[MHz], corresponding to a clock period of 10[ns], this indicates that the data path delay comfortably meets the timing requirements and it equals to $10[ns] - 3.103[ns] = 6.897[ns]$.

12. FPGA programming and debugging

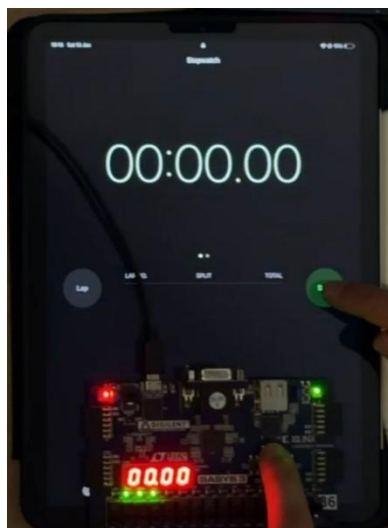
screenshot of Project Summary screen including post-implementation Timing, Power, Utilization details:



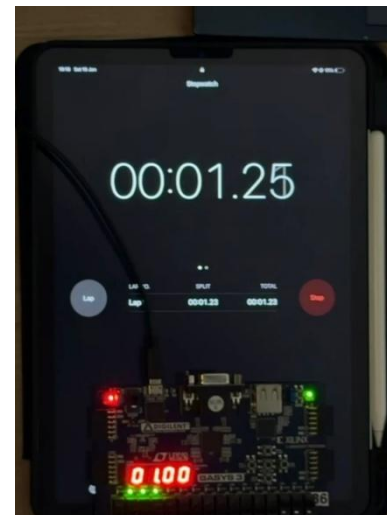
Providing images of our working Project:

We ran two main tests on the system. First, we checked the stopwatch features to ensure the timer worked correctly. Then, we performed a second test to confirm that the Stash module was successfully capturing and saving the time from the counter.

Test 1:



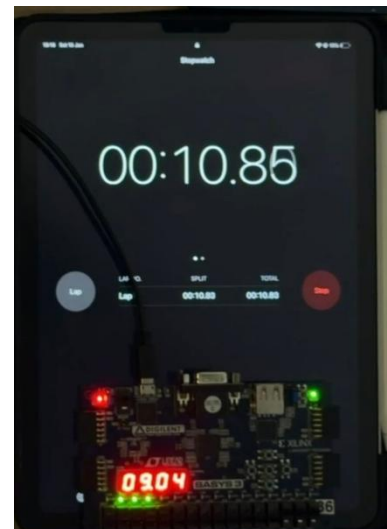
RESET



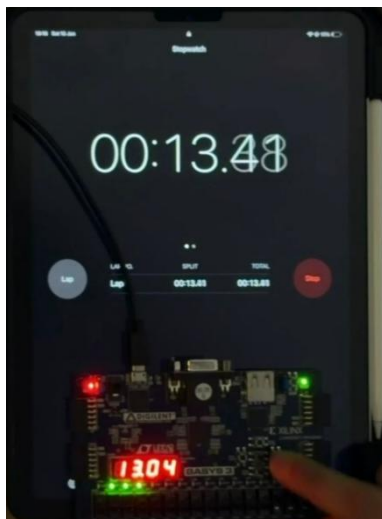
Press Trigger



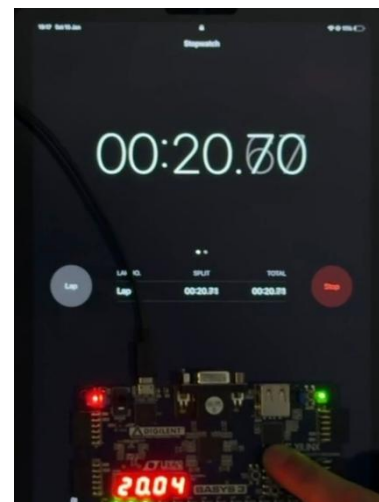
Press Sample



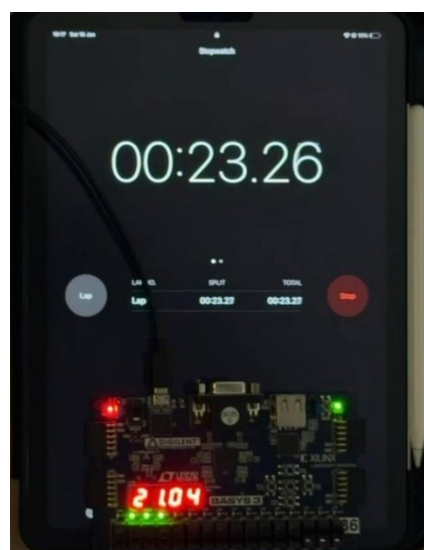
Press Split



Press Split again

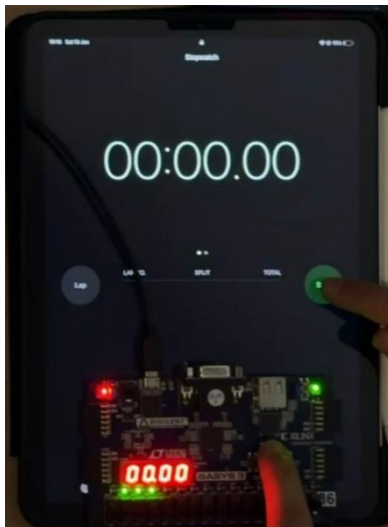


Press 2 triggers (go to paused state)
we made 2 seconds delay

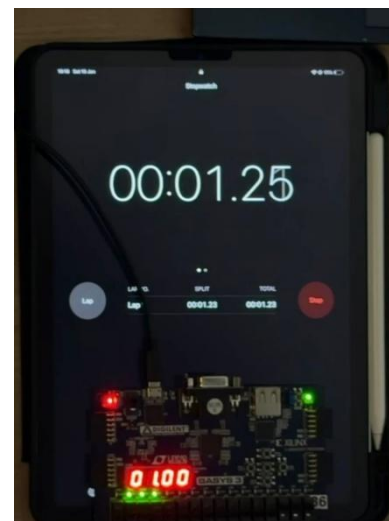


Press trigger (Counting state)

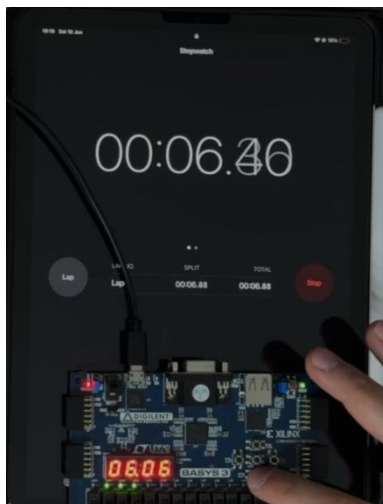
Test 2:



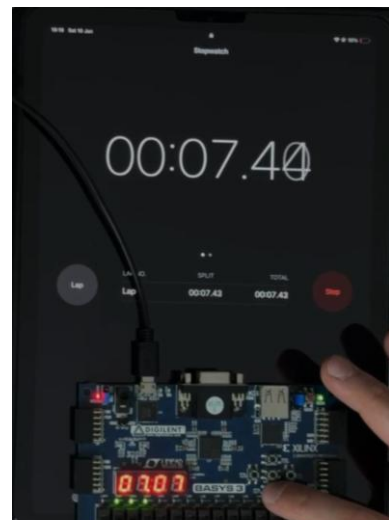
Press Reset



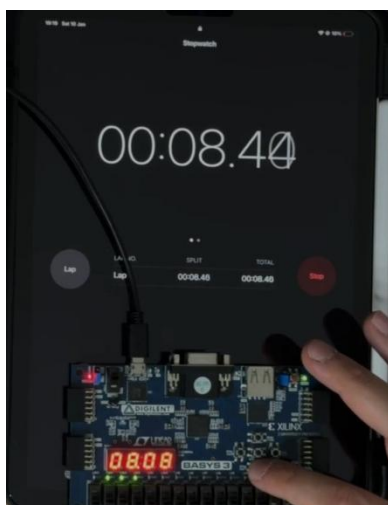
Press Trigger



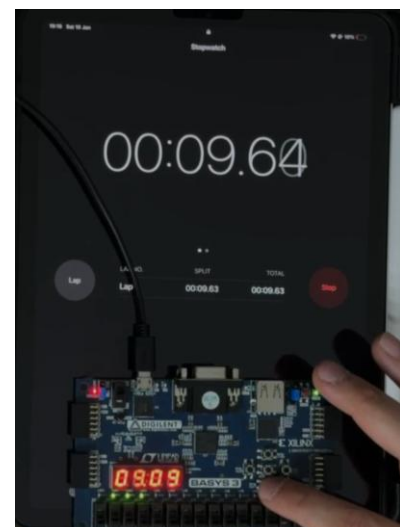
Press Sample



Prsee Sample



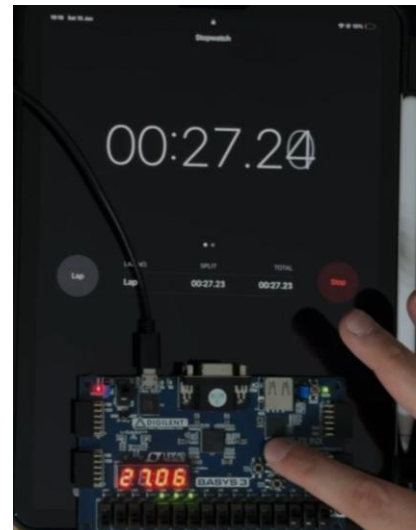
Prsee Sample



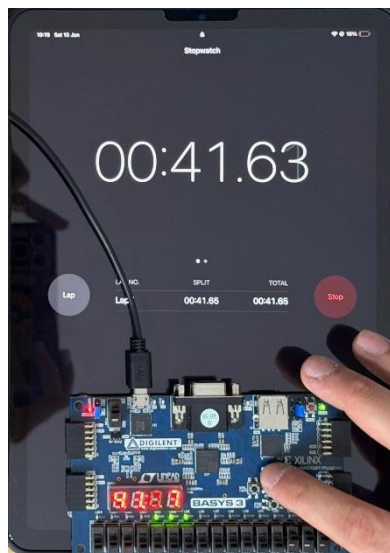
Press Sample



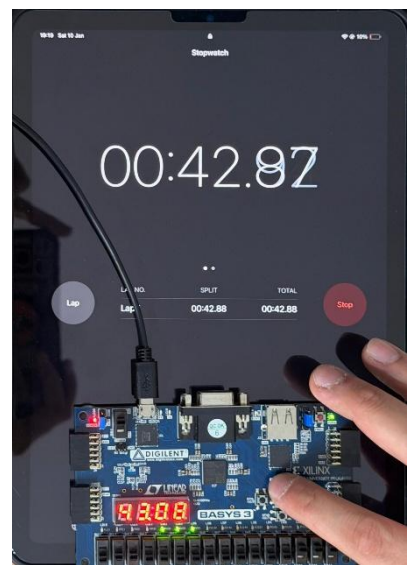
Press Toggle



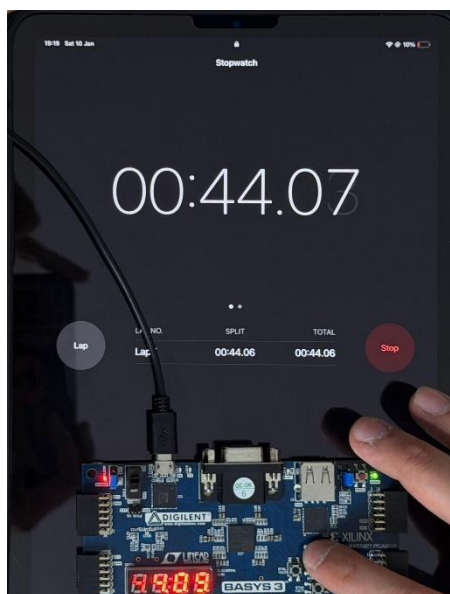
Press Trigger/Next sample



Press Trigger/Next sample



Press Trigger/Next sample



Press Trigger/Next sample

We can see that the attaches images confirm that our project works properly showing the active stopwatch timer on the two left most digits and the stored stash value on the two right most digits.